

FUNCTIONS

Scenario 1: CalculateAge

```
CREATE OR REPLACE FUNCTION CalculateAge(  
    p_DOB DATE  
)  
RETURN NUMBER  
AS  
    v_Age NUMBER;  
BEGIN  
    v_Age := TRUNC(MONTHS_BETWEEN(SYSDATE, p_DOB) / 12);  
    RETURN v_Age;  
END;  
/  
  
SELECT CalculateAge(TO_DATE('1985-05-15','YYYY-MM-DD')) AS Age  
FROM DUAL;
```

FreeSQL >_ Worksheet SQL Tutorials

Navigator Files

My Schema

Tables

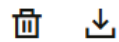
Search

- ACCOUNTS
- CUSTOMERS
- EMPLOYEES
- LOANS
- TRANSACTIONS

[SQL Worksheet]*      Aa 

```
1 CREATE OR REPLACE FUNCTION CalculateAge(  
2     p_DOB DATE  
3 )  
4 RETURN NUMBER  
5 AS  
6     v_Age NUMBER;  
7 BEGIN  
8     v_Age := TRUNC(MONTHS_BETWEEN(SYSDATE, p_DOB) / 12);  
9     RETURN v_Age;  
10 END;  
11 /
```

Query result Script output DBMS output Explain Plan SC



```
SQL> CREATE OR REPLACE FUNCTION CalculateAge(  
      p_DOB DATE  
      )  
      RETURN NUMBER...  
Show more...
```

Function CALCULATEAGE compiled

Elapsed: 00:00:00.023

FreeSQL > Worksheet SQL Tutorials

Navigator Files

My Schema

Tables

Search

ACCOUNTS

CUSTOMERS

EMPLOYEES

LOANS

TRANSACTIONS

[SQL Worksheet]*

```

1 SELECT CalculateAge (TO_DATE ('1985-05-15', 'YYYY-MM-DD')) AS Age
2 FROM DUAL;

```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.004 seconds

	AGE
1	41

Scenario 2: CalculateMonthlyInstallment

```

CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(
    p_LoanAmount NUMBER,
    p_InterestRate NUMBER,
    p_Years NUMBER
)
RETURN NUMBER
AS
    v_TotalAmount NUMBER;
    v_MonthlyInstallment NUMBER;
BEGIN
    v_TotalAmount := p_LoanAmount + (p_LoanAmount * p_InterestRate / 100);
    v_MonthlyInstallment := v_TotalAmount / (p_Years * 12);

    RETURN ROUND(v_MonthlyInstallment,2);
END;
/

```

SELECT CalculateMonthlyInstallment(50000,8,5) AS MonthlyInstallment
FROM DUAL;

The screenshot displays the FreeSQL web application interface. At the top, there are tabs for 'FreeSQL', '>_ Worksheet', and 'SQL Tutorials'. On the left, a 'Navigator' sidebar shows a tree view with 'My Schema' and 'Tables'. Below this is a search bar and a list of database objects: ACCOUNTS, CUSTOMERS, EMPLOYEES, LOANS, and TRANSACTIONS. The main area is a SQL editor titled '[SQL Worksheet]*' with a toolbar containing icons for execution, undo, redo, and other functions. The editor contains the following SQL code:

```
1 CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(  
2     p_LoanAmount NUMBER,  
3     p_InterestRate NUMBER,  
4     p_Years NUMBER  
5 )  
6 RETURN NUMBER  
7 AS  
8     v_TotalAmount NUMBER;  
9     v_MonthlyInstallment NUMBER;  
10 BEGIN  
11     v_TotalAmount := p_LoanAmount + ((p_LoanAmount * p_InterestRate / 100));  
12     v_MonthlyInstallment := v_TotalAmount / (p_Years * 12);  
13  
14     RETURN ROUND(v_MonthlyInstallment,2);  
15 END;  
16 /
```

Below the editor, there are tabs for 'Query result', 'Script output', 'DBMS output', 'Explain Plan', and 'SQL history'. The 'Script output' tab is active, showing the execution results. It includes a 'Show more...' link and the following output:

```
SQL> CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment(  
      p_LoanAmount NUMBER,  
      p_InterestRate NUMBER,  
      p_Years NUMBER...  
Show more...
```

Below the output, it states: 'Function CALCULATEMONTHLYINSTALLMENT compiled' and 'Elapsed: 00:00:00.023'.

FreeSQL > Worksheet SQL Tutorials

Navigator Files

My Schema

Tables

Search

ACCOUNTS

CUSTOMERS

EMPLOYEES

LOANS

TRANSACTIONS

[SQL Worksheet]*

```

1 SELECT CalculateMonthlyInstallment(50000,8,5) AS MonthlyInstallment
2 FROM DUAL;

```

Query result Script output DBMS output Explain Plan SQL history

Download Execution time: 0.007 seconds

	MONTHLYINSTALL
1	900

Scenario 3: HasSufficientBalance

```

CREATE OR REPLACE FUNCTION HasSufficientBalance(
  p_AccountID NUMBER,
  p_Amount NUMBER
)
RETURN BOOLEAN
AS
  v_Balance NUMBER;
BEGIN
  SELECT Balance
  INTO v_Balance
  FROM Accounts
  WHERE AccountID = p_AccountID;

```

```

    RETURN v_Balance >= p_Amount;
END;
/

SET SERVEROUTPUT ON;

DECLARE
    v_Result BOOLEAN;
BEGIN
    v_Result := HasSufficientBalance(102,5000);

    IF v_Result THEN
        DBMS_OUTPUT.PUT_LINE('Sufficient Balance');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
    END IF;
END;
/

```

Cognizant GenC Ne... Free JavaScript Cour... Uninstall VPN Brow... Cognizant Skillspring

FreeSQL >_ Worksheet SQL Tutorials

Navigator Files

My Schema

Tables

Search

ACCOUNTS

CUSTOMERS

EMPLOYEES

LOANS

TRANSACTIONS

[SQL Worksheet]*

```

1 CREATE OR REPLACE FUNCTION HasSufficientBalance (
2     p_AccountID NUMBER,
3     p_Amount NUMBER
4 )
5 RETURN BOOLEAN
6 AS
7     v_Balance NUMBER;
8 BEGIN
9     SELECT Balance
10    INTO v_Balance
11   FROM Accounts
12  WHERE AccountID = p_AccountID;
13
14     RETURN v_Balance >= p_Amount;
15 END;
16 /

```

Query result Script output DBMS output Explain Plan SQL history

SQL> CREATE OR REPLACE FUNCTION HasSufficientBalance(
 p_AccountID NUMBER,
 p_Amount NUMBER
)...
[Show more...](#)

Function HASSUFFICIENTBALANCE compiled

Elapsed: 00:00:00.020

FreeSQL >_ Worksheet SQL Tutorials

Navigator Files

My Schema

Tables

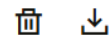
Search

- ACCOUNTS
- CUSTOMERS
- EMPLOYEES
- LOANS
- TRANSACTIONS

[SQL Worksheet]*

```
1 SET SERVEROUTPUT ON;
2
3 DECLARE
4     v_Result BOOLEAN;
5 BEGIN
6     v_Result := HasSufficientBalance(102,5000);
7
8     IF v_Result THEN
9         DBMS_OUTPUT.PUT_LINE('Sufficient Balance');
10    ELSE
11        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');
12    END IF;
13 END;
```

Query result Script output DBMS output Explain Plan SQL history



BEGIN
v_Result := HasSufficientBalance(102,5000);...
[Show more...](#)

Sufficient Balance

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.162